Unidad 5-1

Árboles Binarios



Programación II
Ingeniería en Computación

Juan Ignacio Iturriaga
Marco Viola



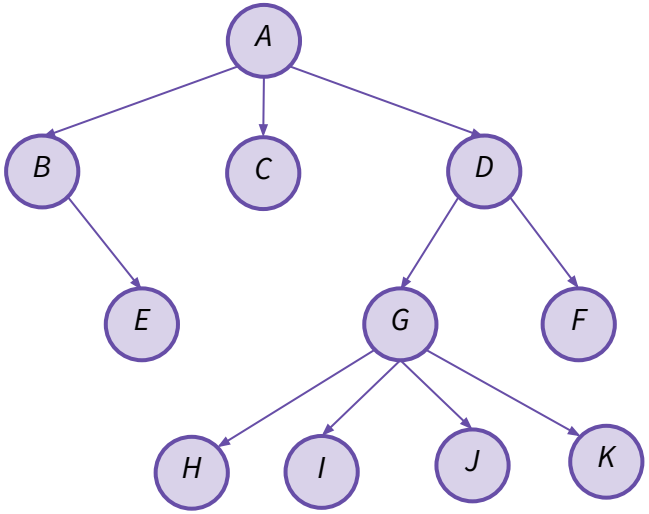
Concepto de árbol

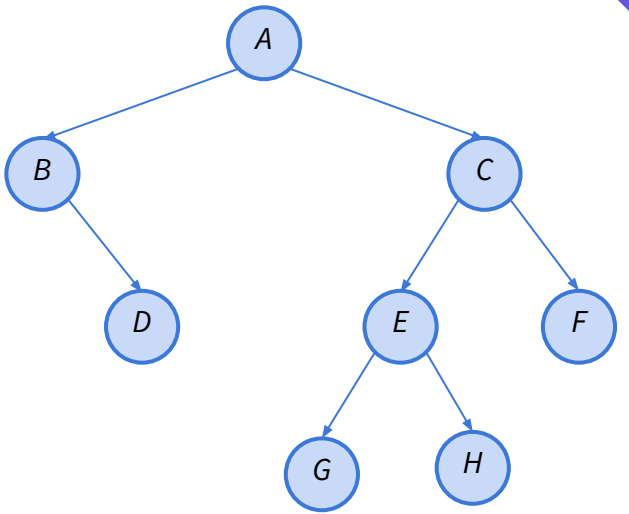


Árboles concepto y propiedades

Trees

- Nodo
- Hijo / Descendiente
- Padre / Ancestro
- Hermano
- **Raíz**
- Hoja
- Subárbol / Rama
- Profundidad / Nivel
- Altura
- Grado

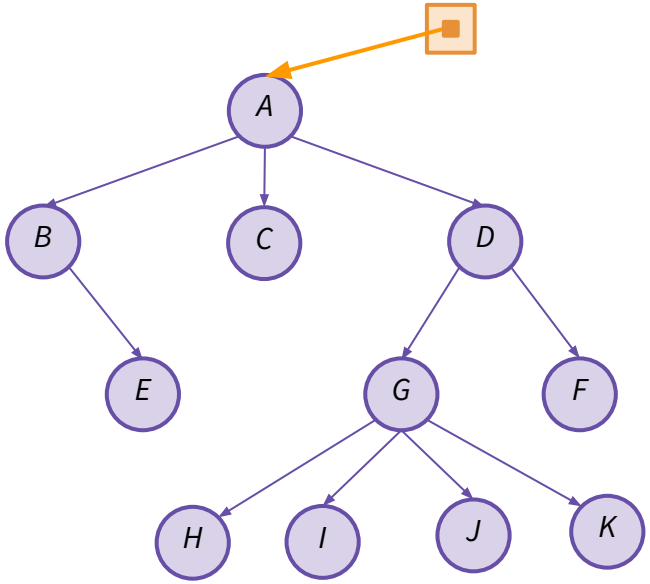


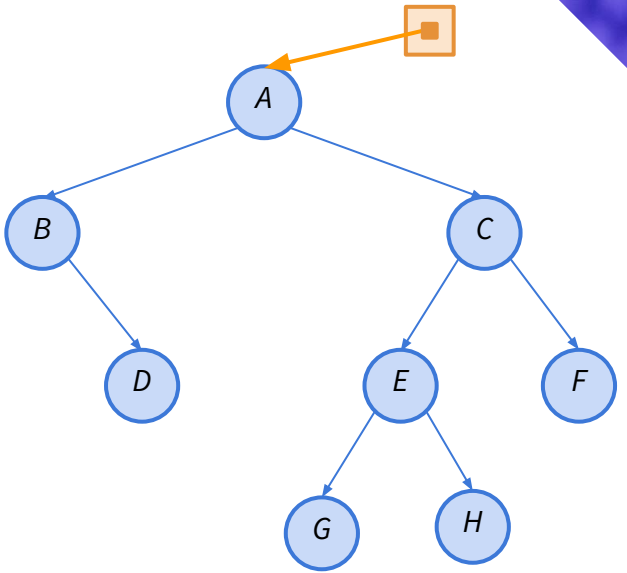


Árboles concepto y propiedades

Trees

- Nodo
- Hijo / Descendiente
- Padre / Ancestro
- Hermano
- **Raíz**
- Hoja
- Subárbol / Rama
- Profundidad / Nivel
- Altura
- Grado

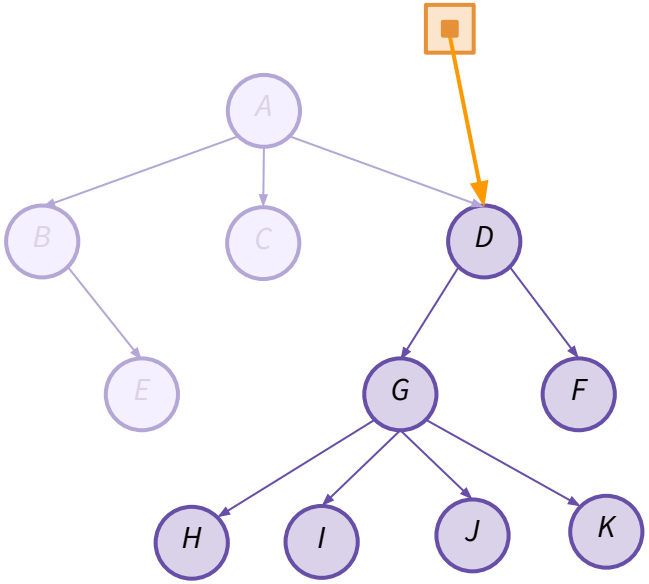


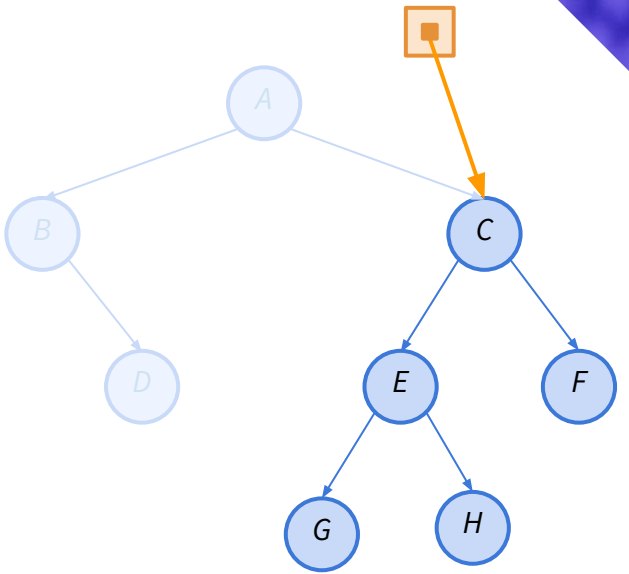


Árboles concepto y propiedades

Trees

- Nodo
- Hijo / Descendiente
- Padre / Ancestro
- Hermano
- **Raíz**
- Hoja
- Subárbol / Rama
- Profundidad / Nivel
- Altura
- Grado





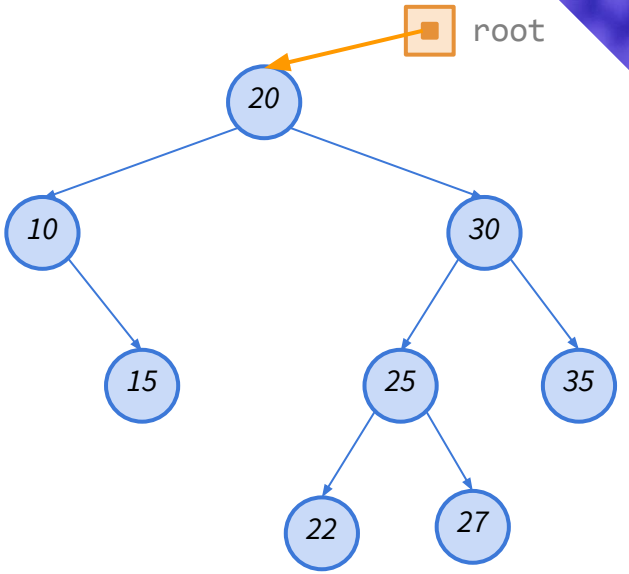


Recorridos



Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

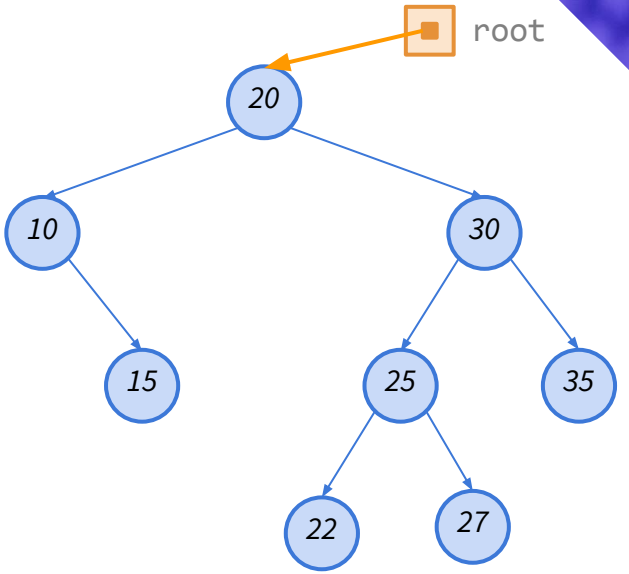



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→

Recorrido

```
void pre_order(btn* root) {
➤     if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

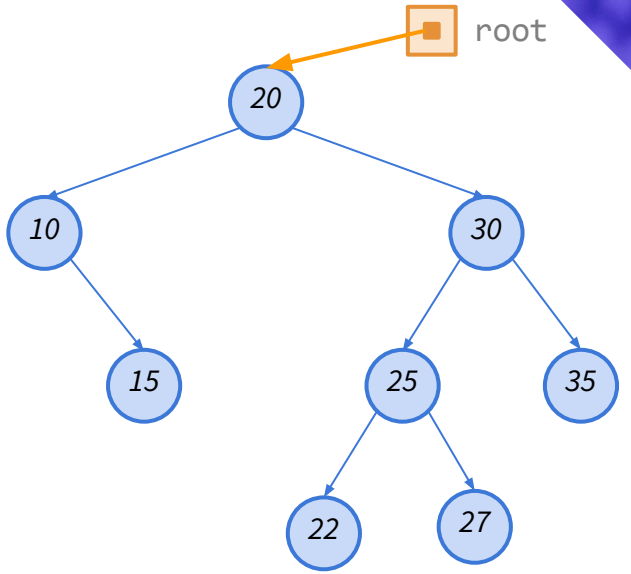



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

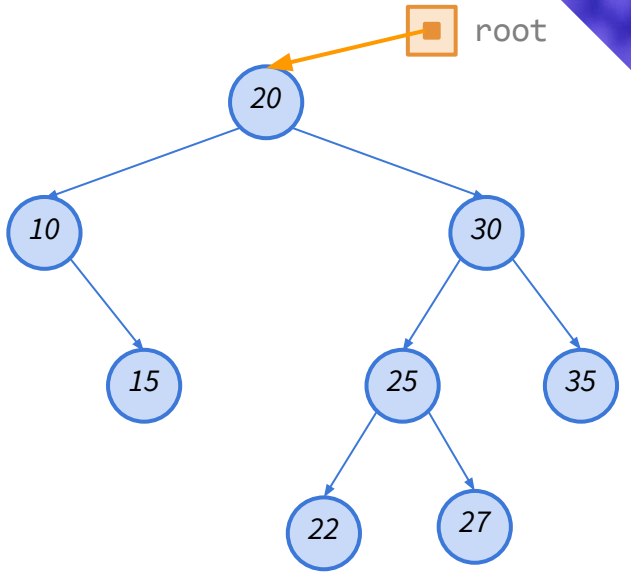



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

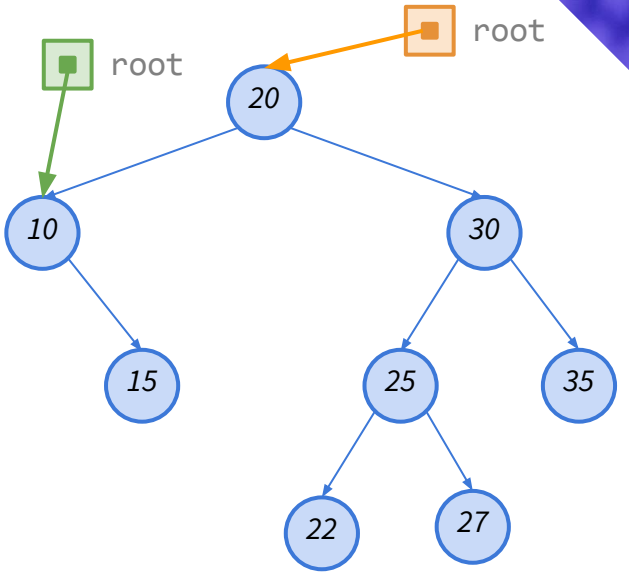



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

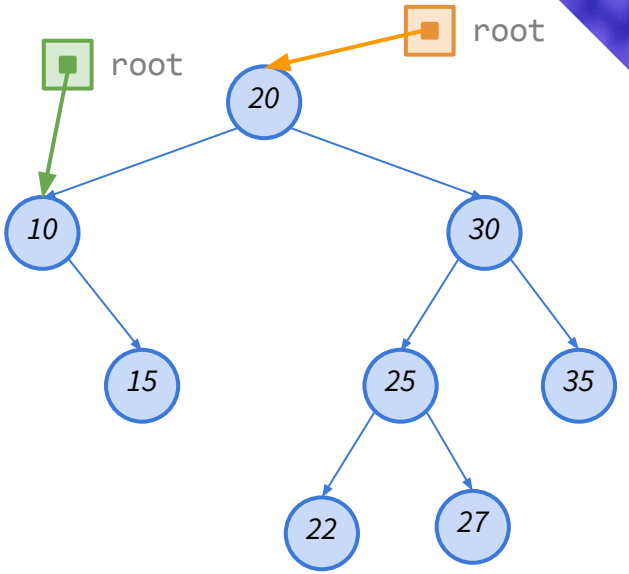



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

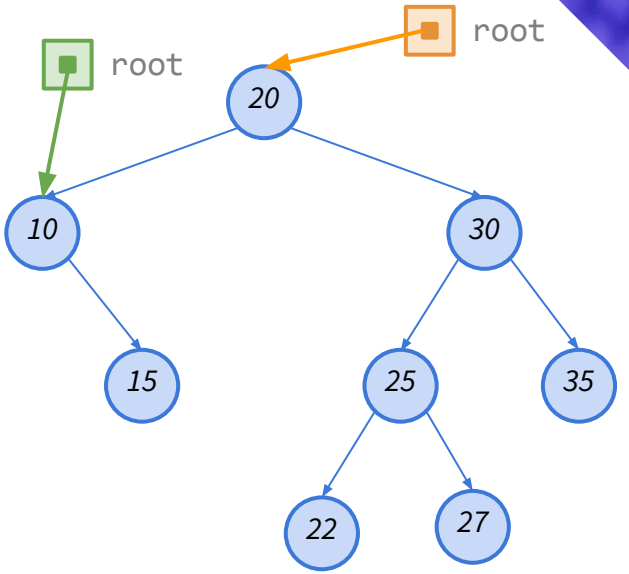



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

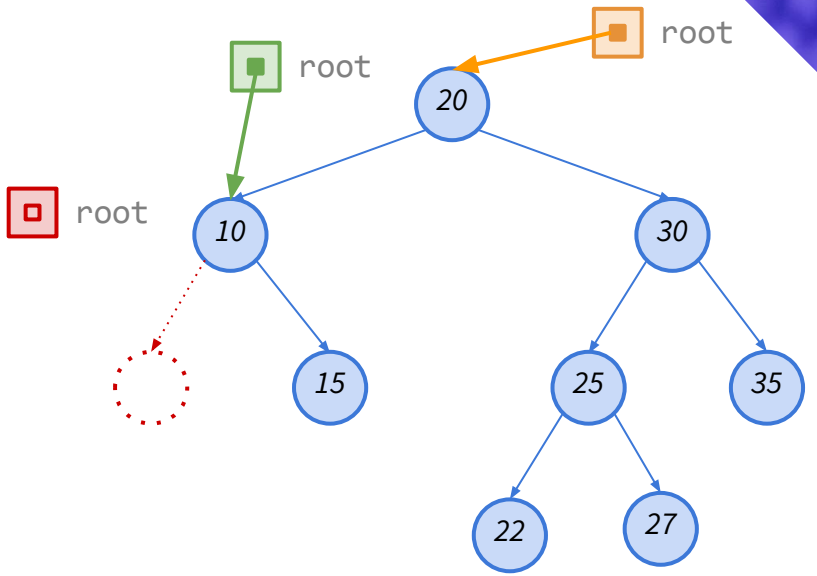



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

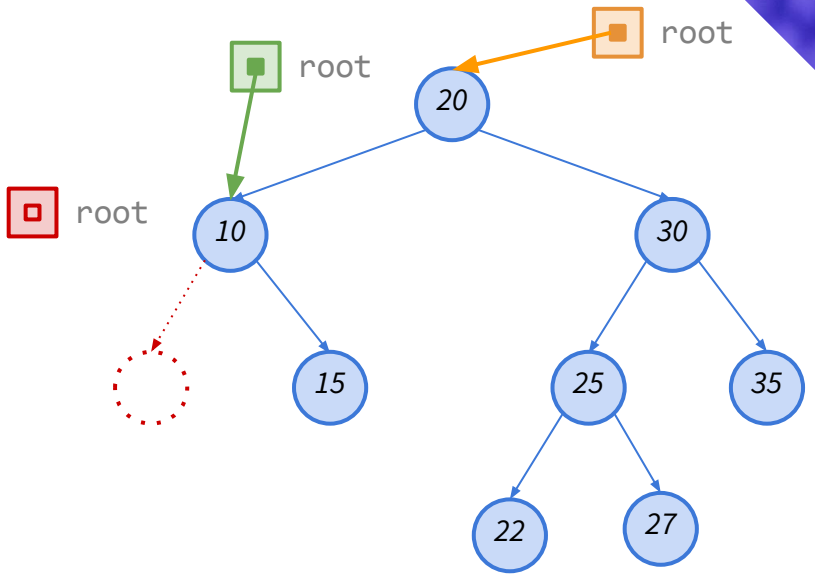



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

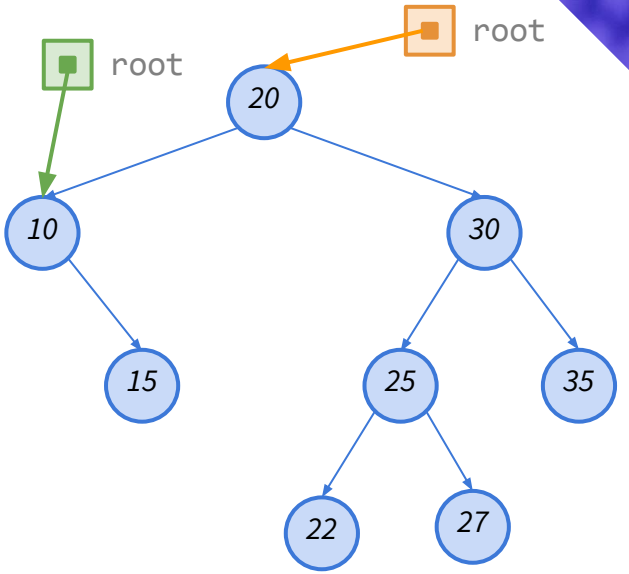



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

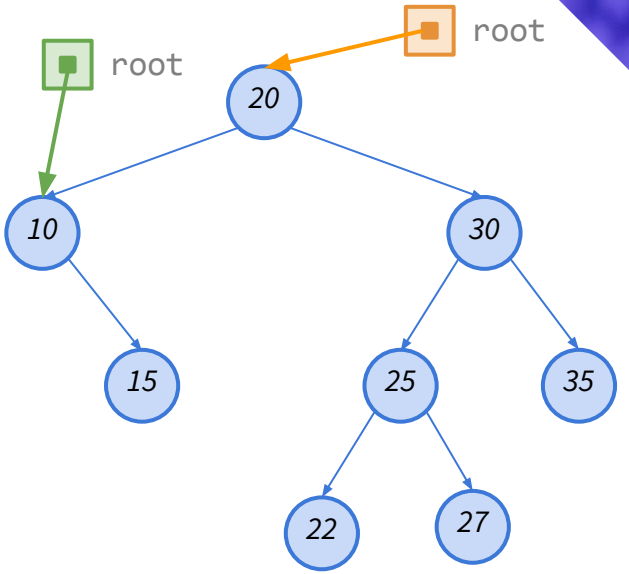



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

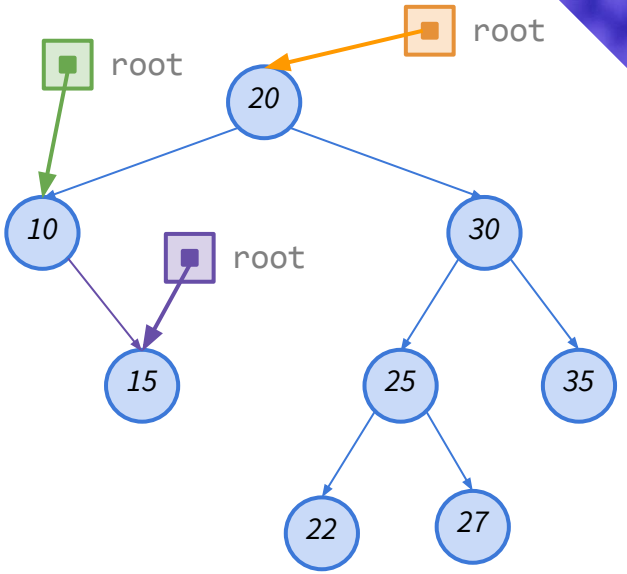



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

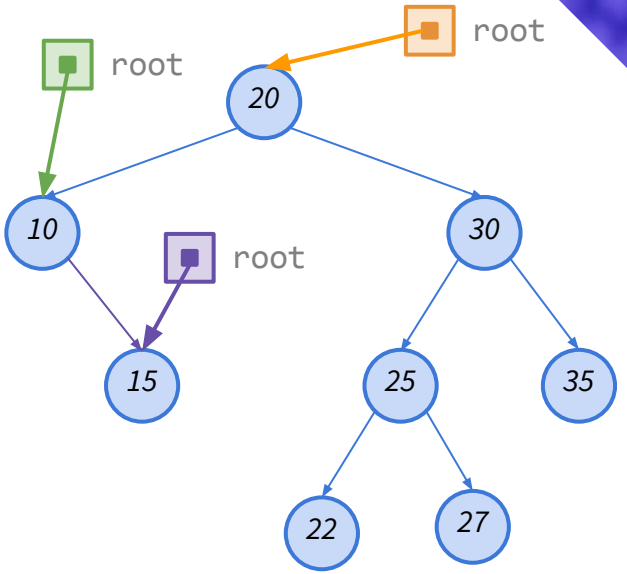


→ 20 10

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

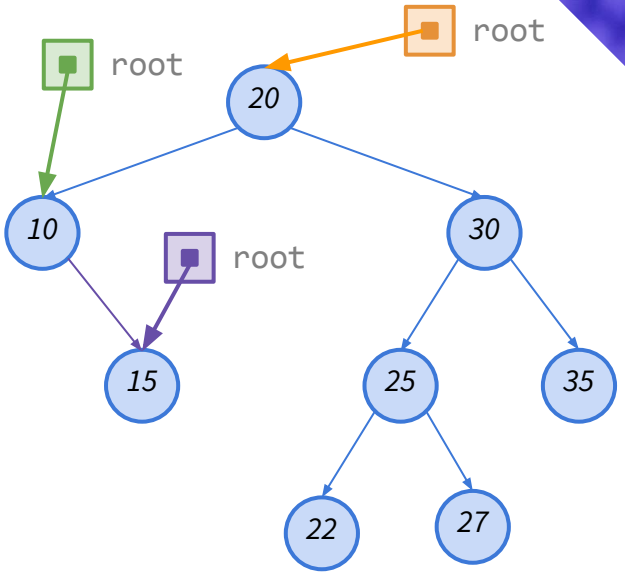



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

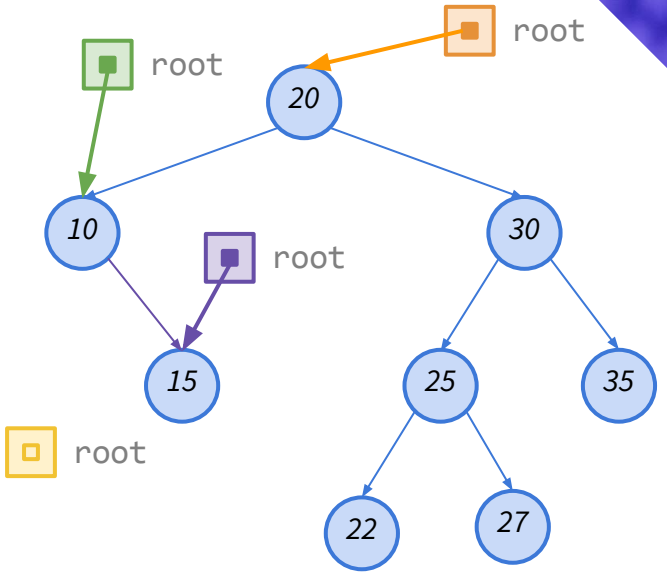



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

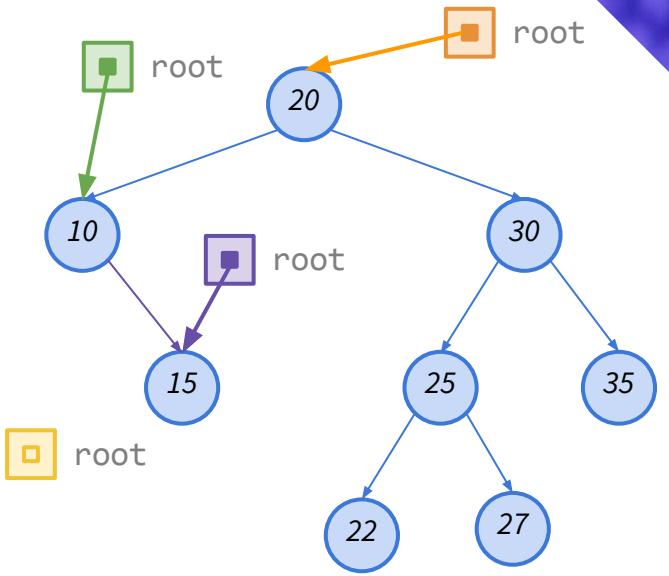


→ 20 10 15

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

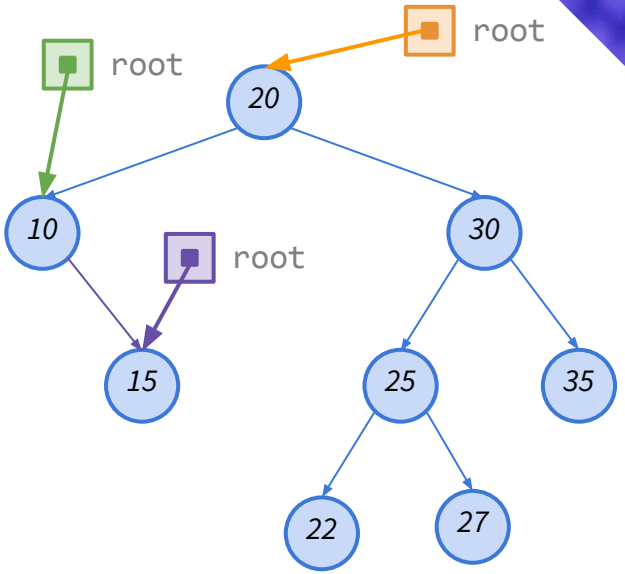



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

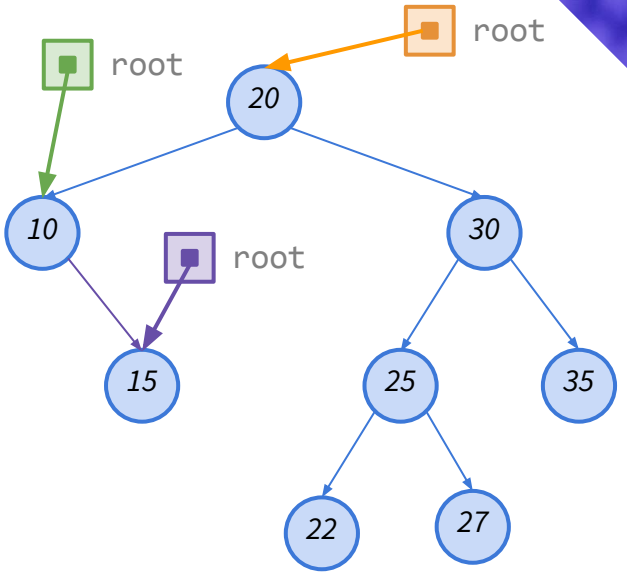



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

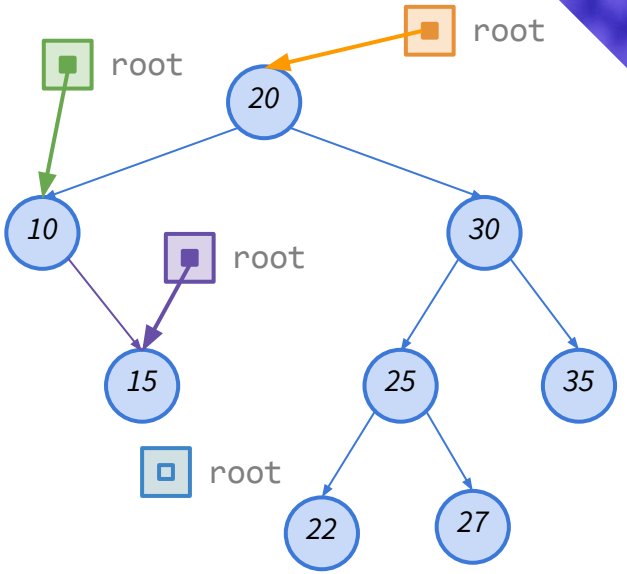



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

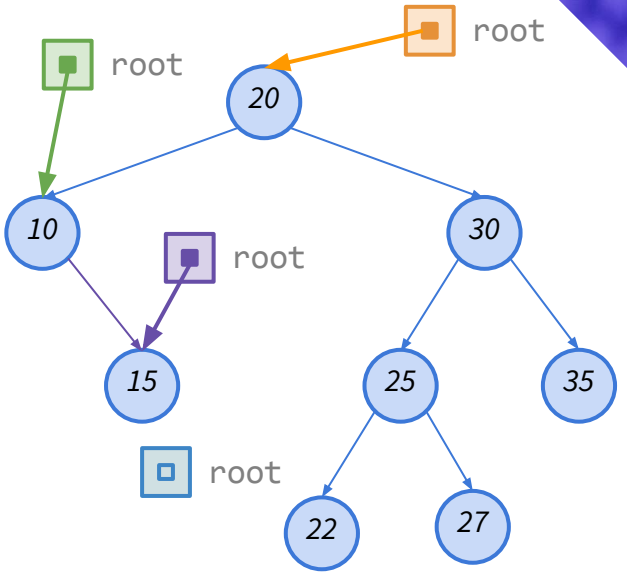



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

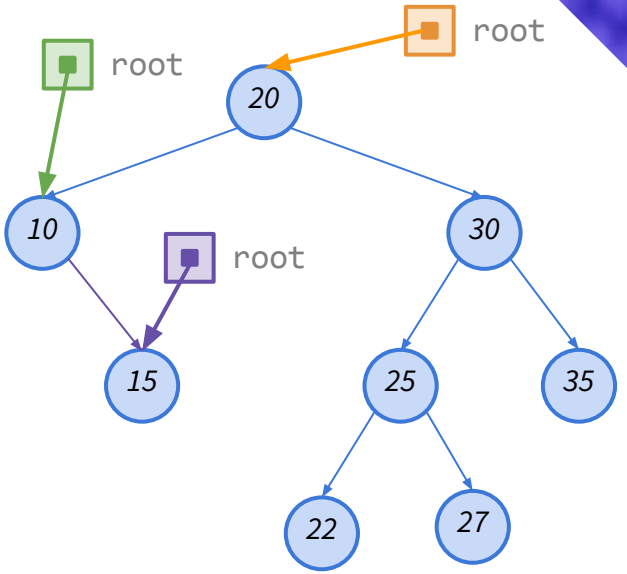



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

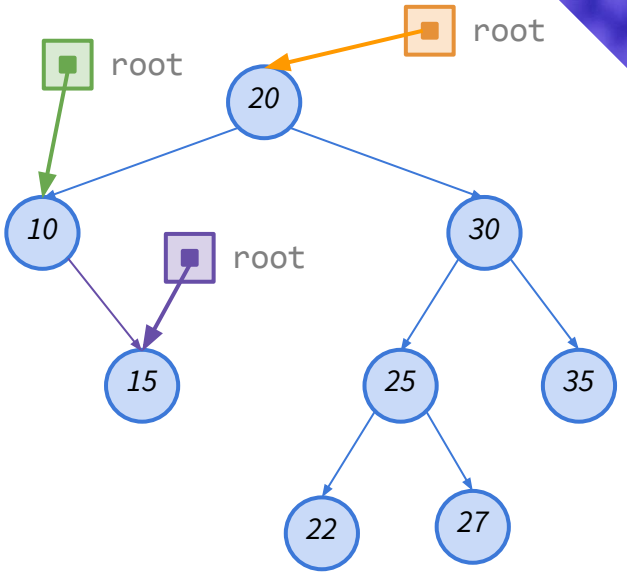



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

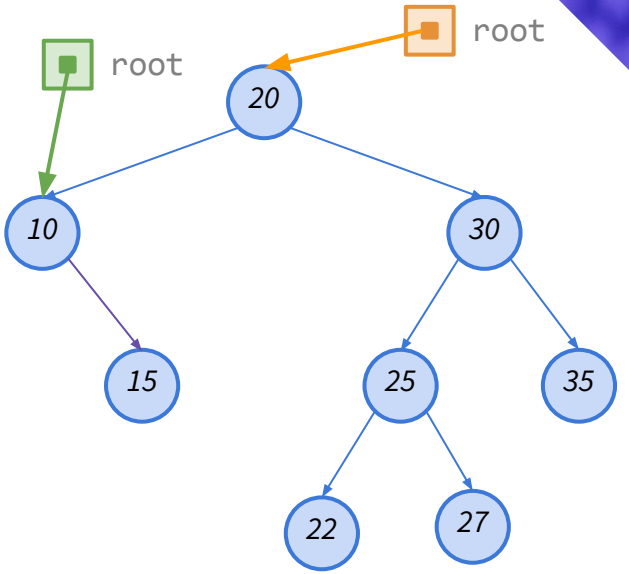



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```c
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

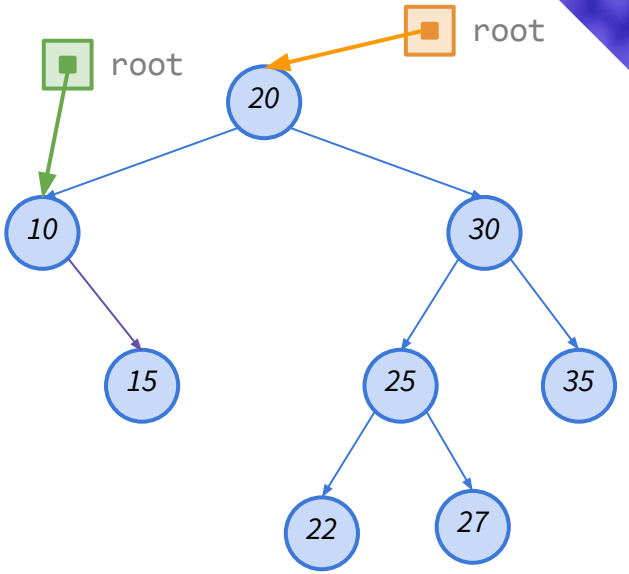



```c
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

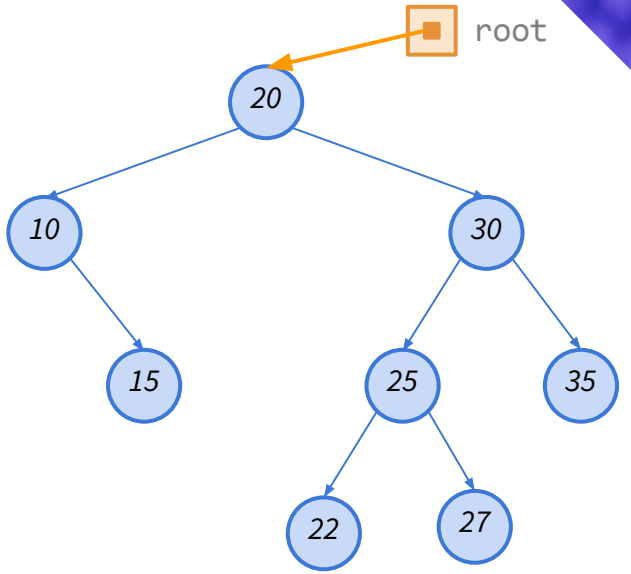



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

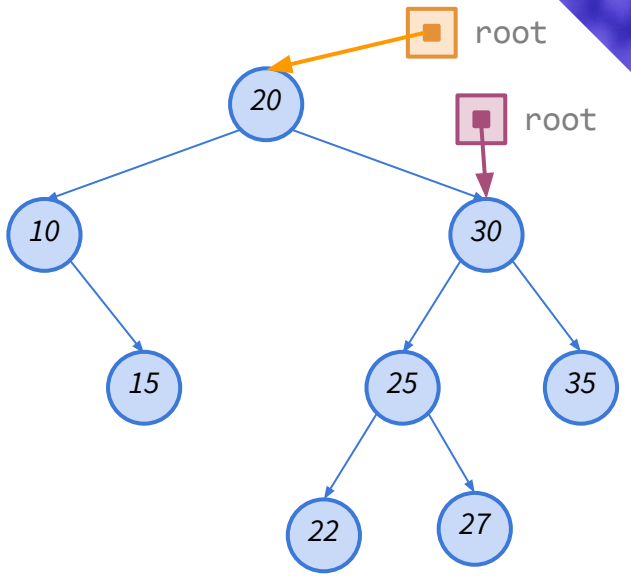



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

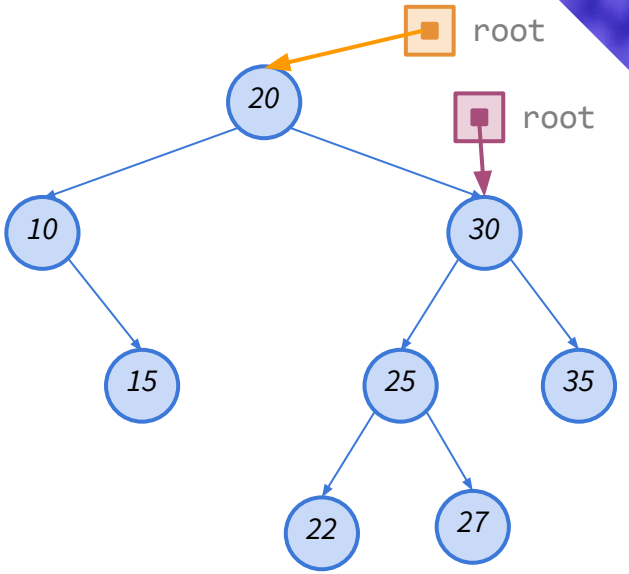



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

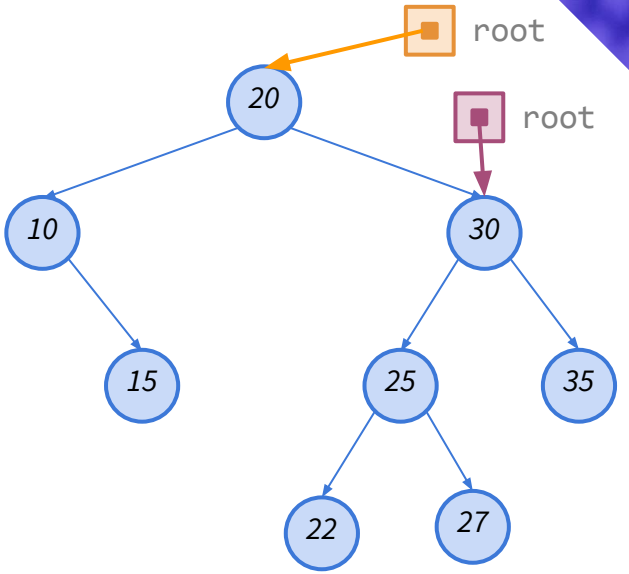



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

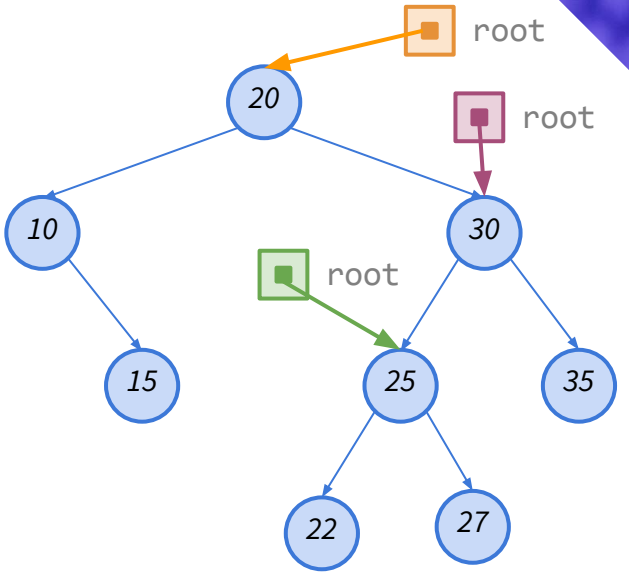



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

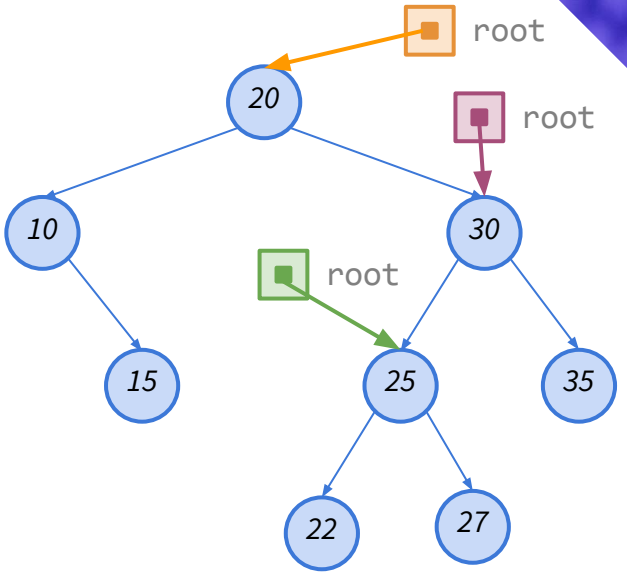



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

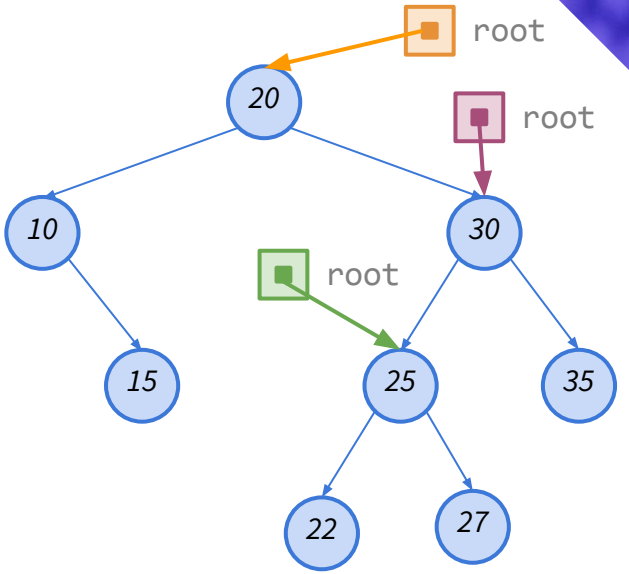


→ 20 10 15 30 25

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

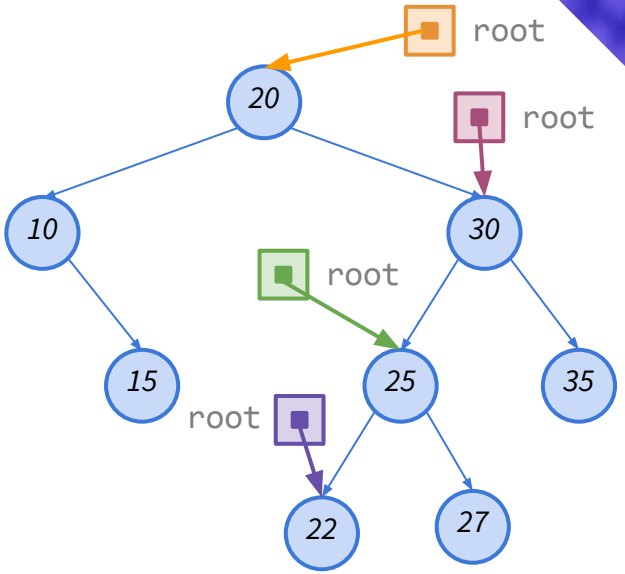


→ 20 10 15 30 25

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

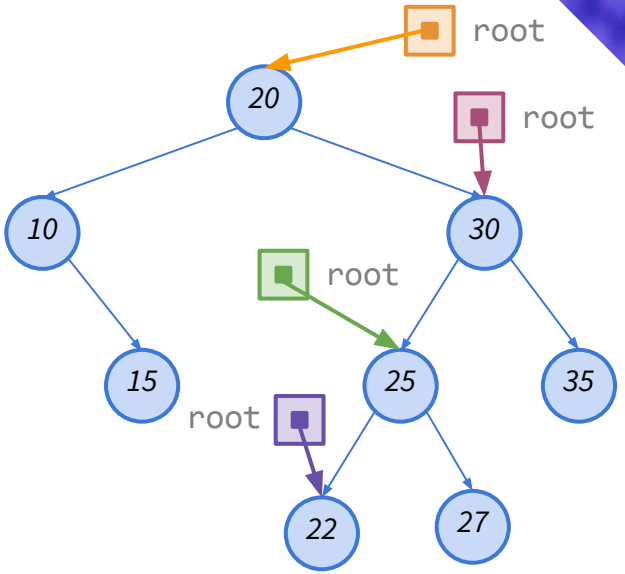



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

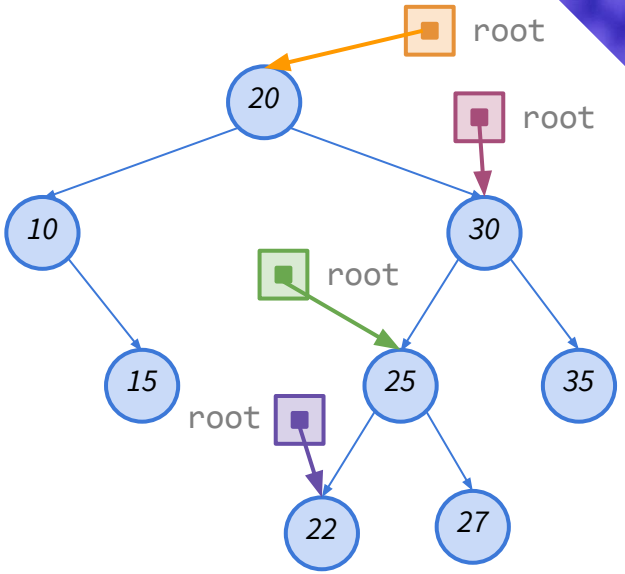



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

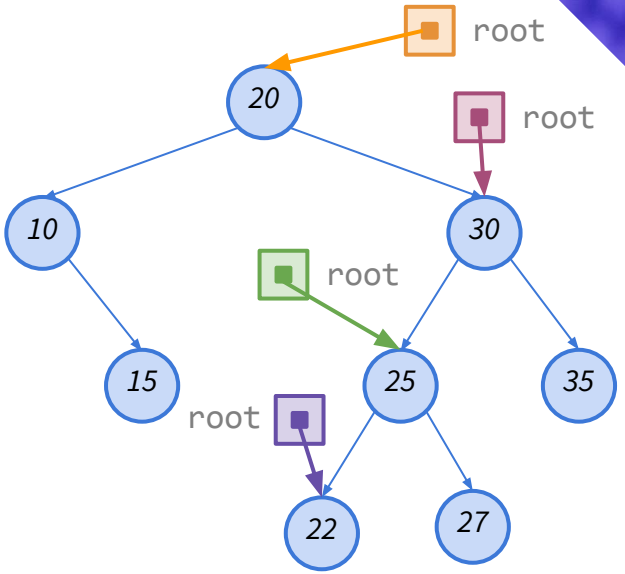


→ 20 10 15 30 25 22

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

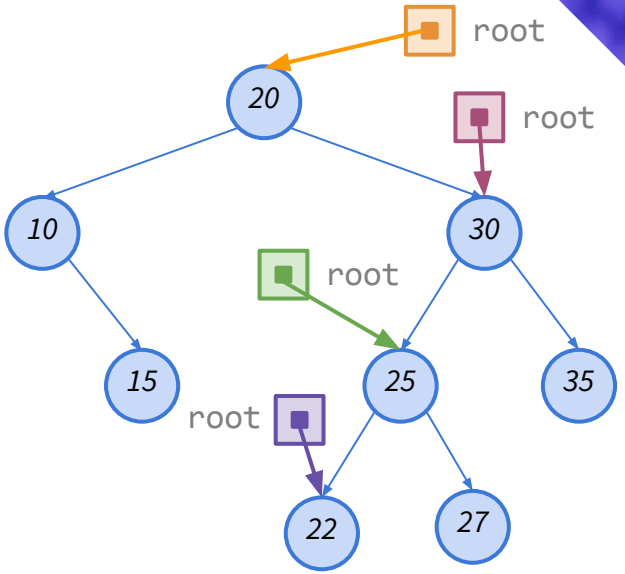



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

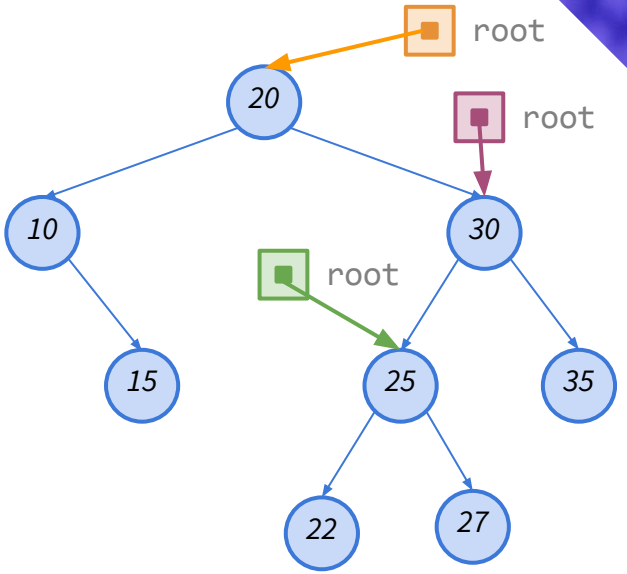


→ 20 10 15 30 25 22

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

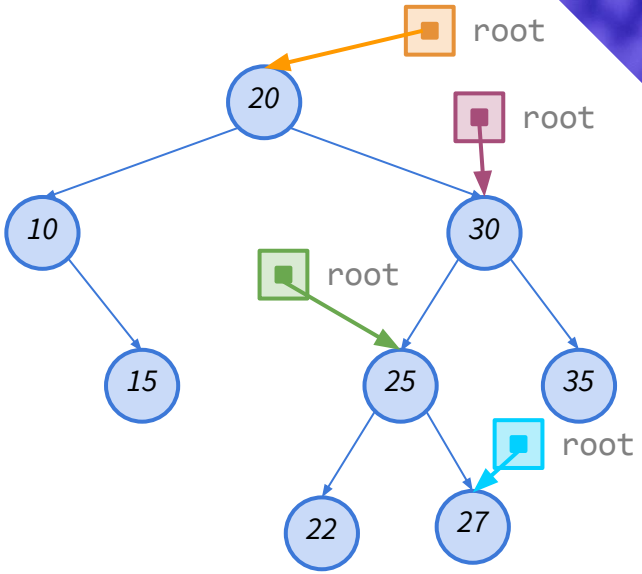



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

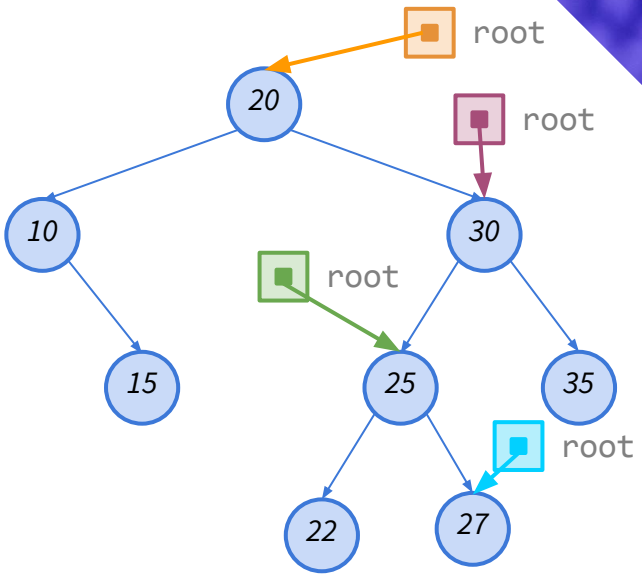


→ 20 10 15 30 25 22

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

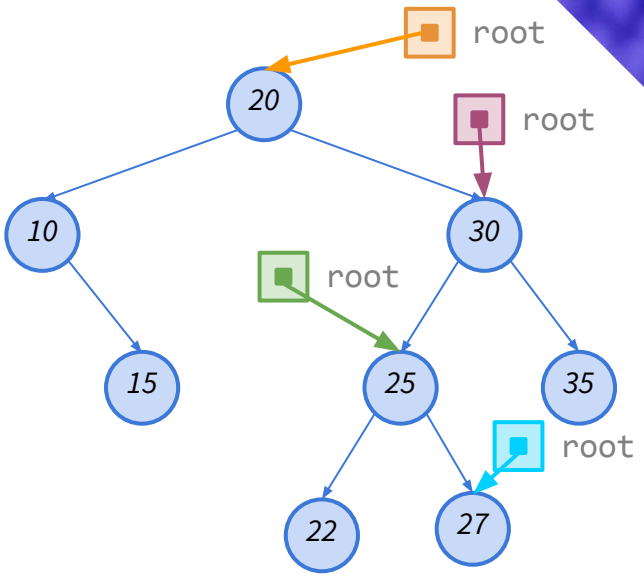



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
        if (root != NULL){
            print_data(root->data);
            pre_order(root->left);
            pre_order(root->right);
        }
}
```

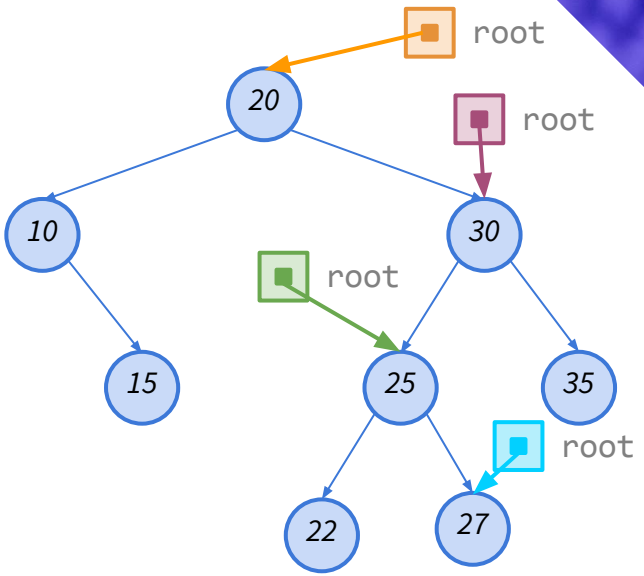



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

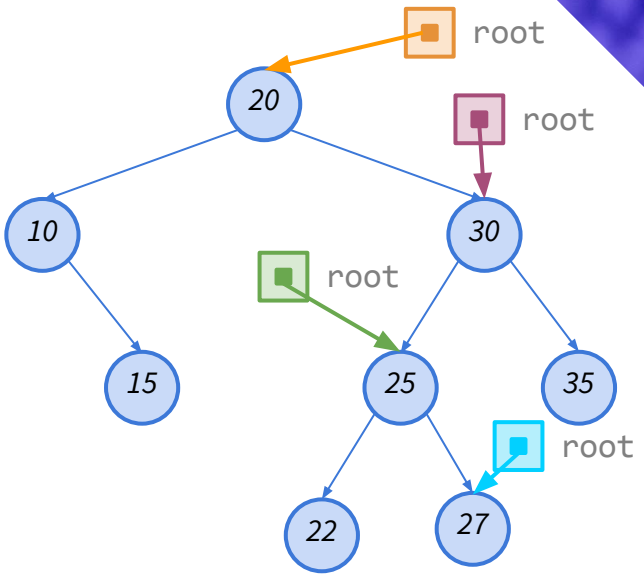


→ 20 10 15 30 25 22 27

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

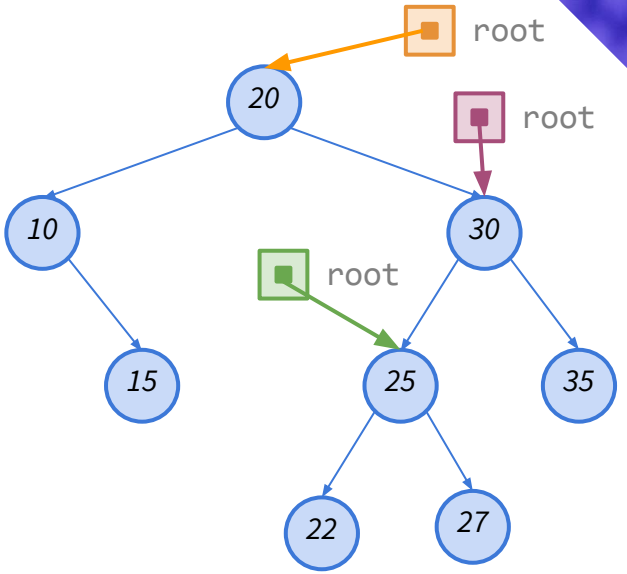



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

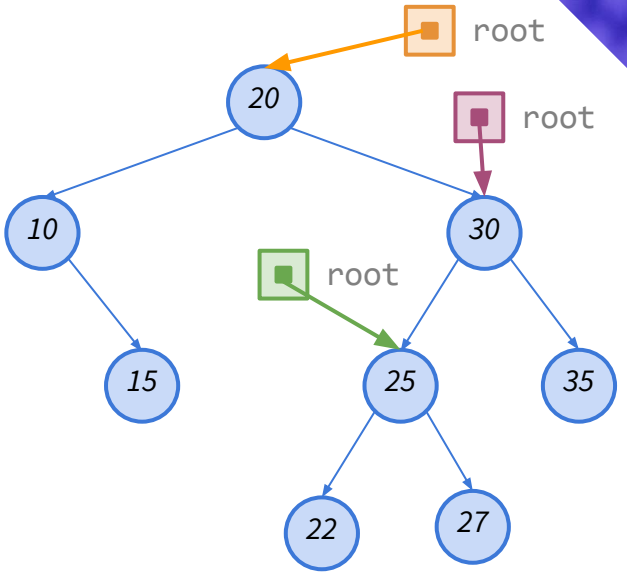


→ 20 10 15 30 25 22 27

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

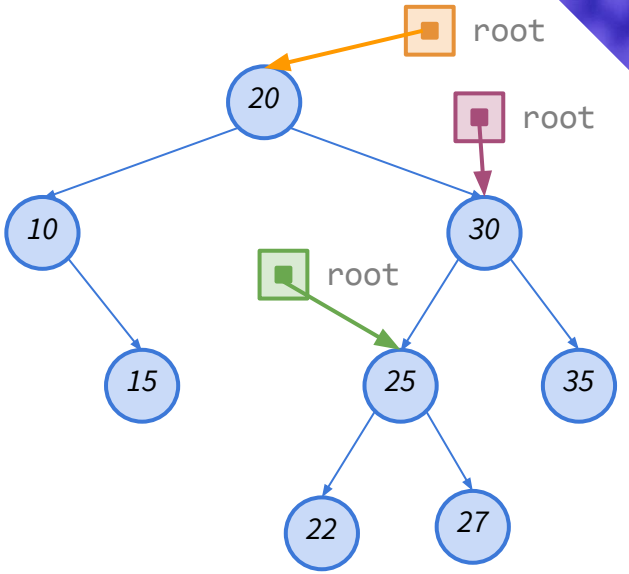



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

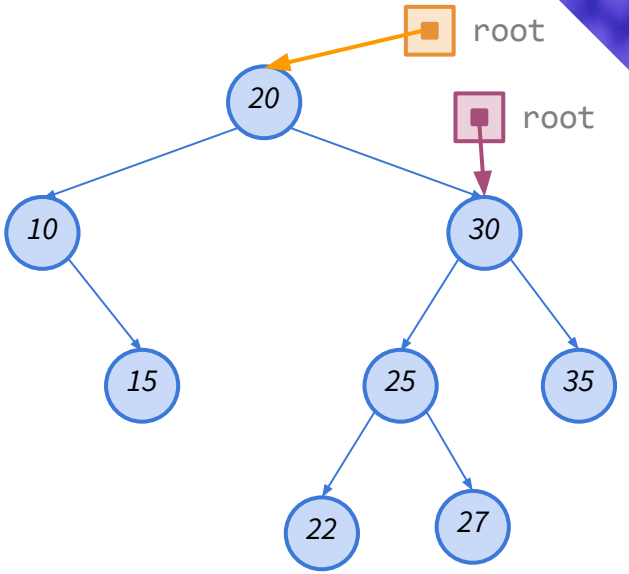



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

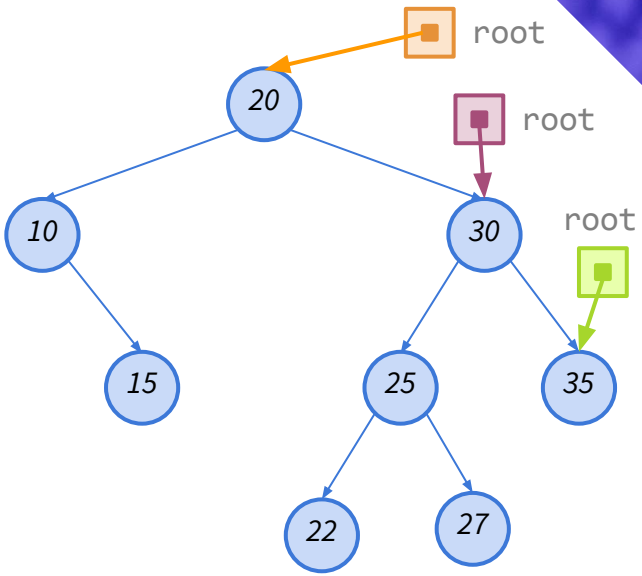



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27

Recorrido

```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

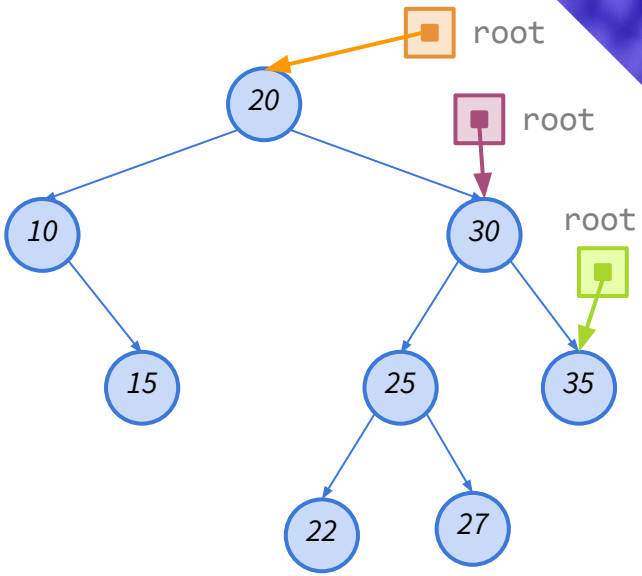


→ 20 10 15 30 25 22 27

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

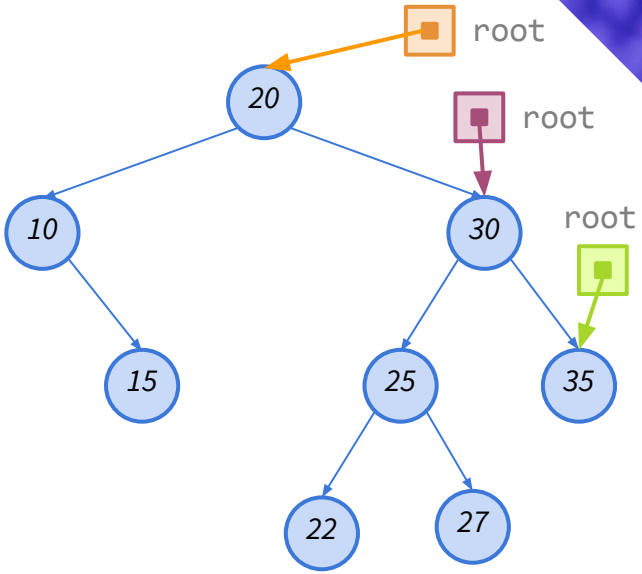


→ 20 10 15 30 25 22 27 35

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

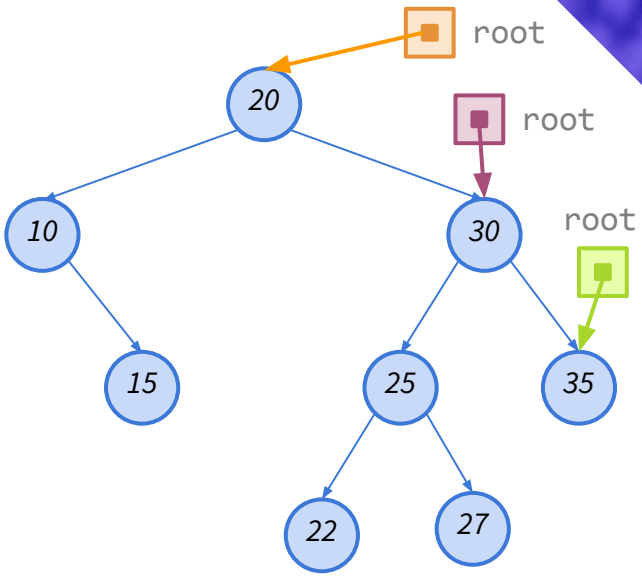


→ 20 10 15 30 25 22 27 35

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

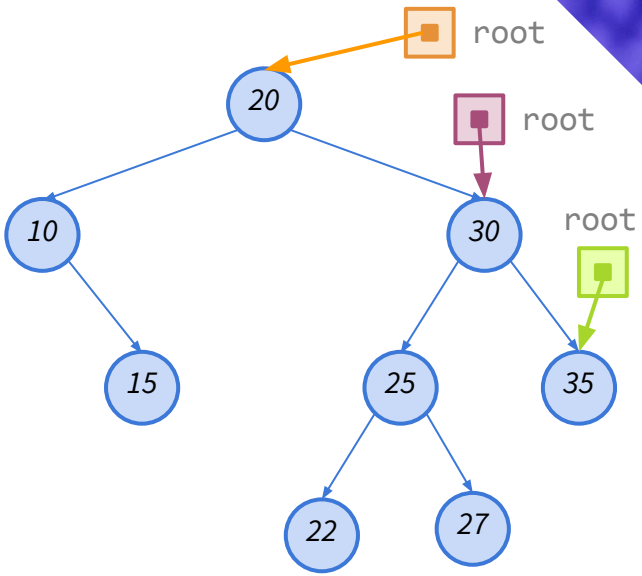


→ 20 10 15 30 25 22 27 35

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
         print_data(root->data);
         pre_order(root->left);
         pre_order(root->right);
      }
}
```

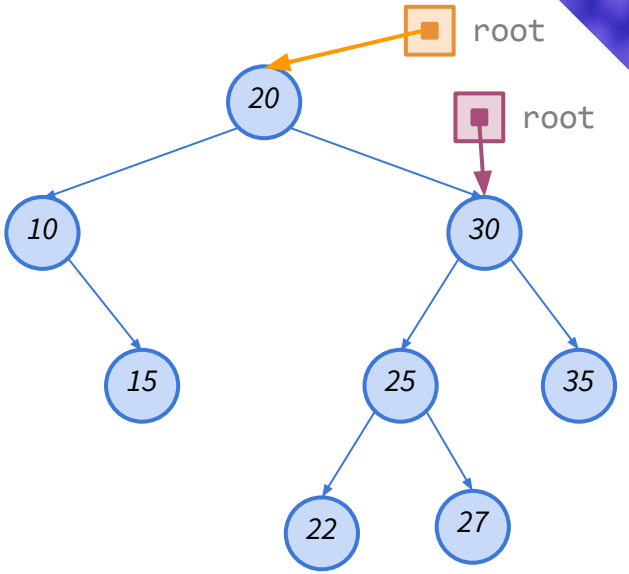


→ 20 10 15 30 25 22 27 35

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

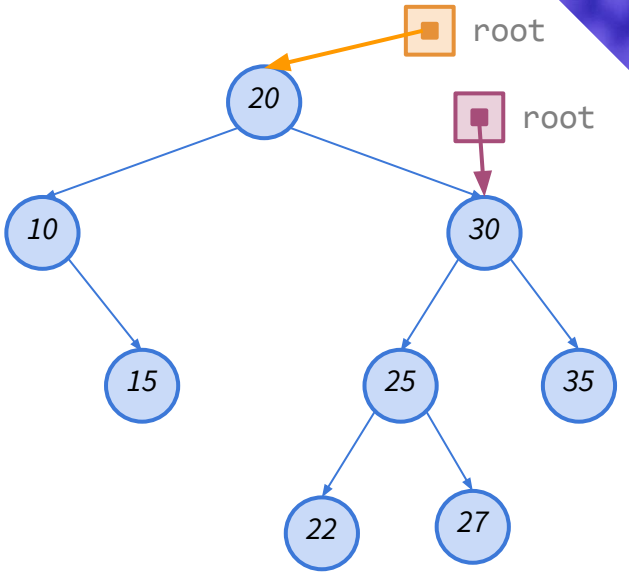



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27 35

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

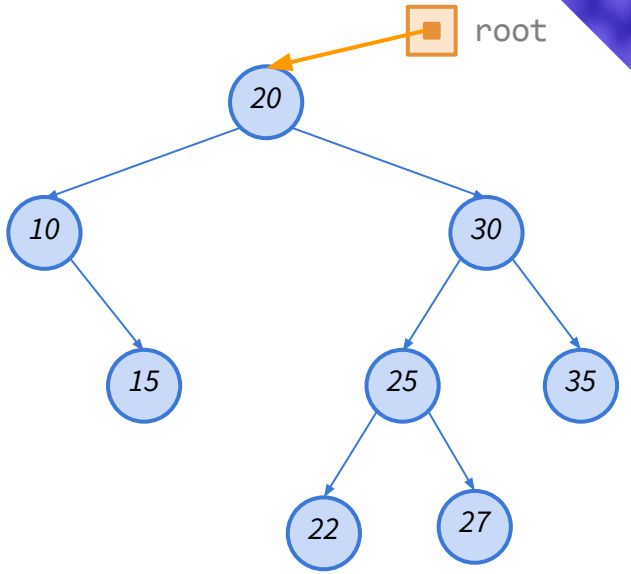



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27 35

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

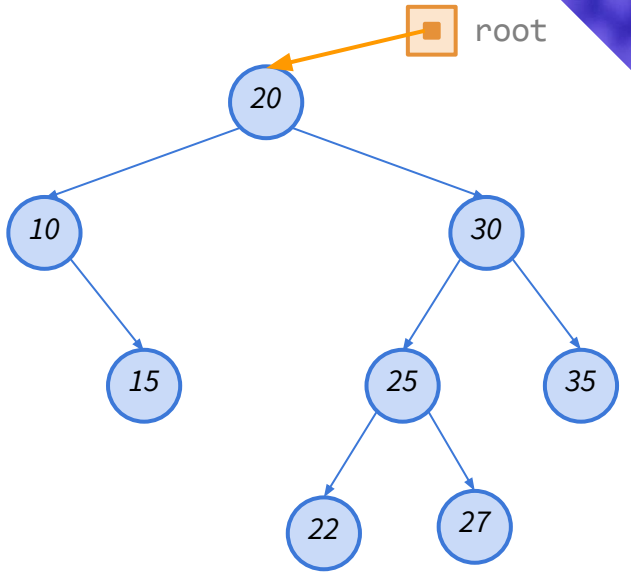



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27 35

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```

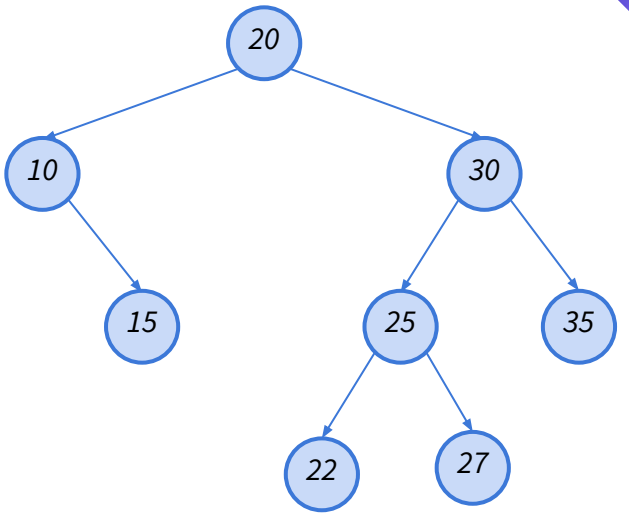



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27 35

Recorrido

```
void pre_order(btn* root) {
      if (root != NULL){
          print_data(root->data);
          pre_order(root->left);
          pre_order(root->right);
      }
}
```




```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

→ 20 10 15 30 25 22 27 35

Recorridos




```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

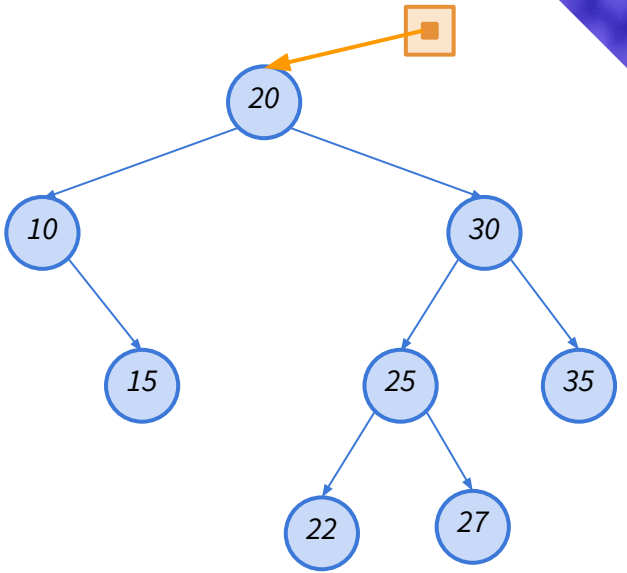
→ 20 10 15 30 25 22 27 35




```
void post_order(btn* root) {
    if (root != NULL){
        post_order(root->left);
        post_order(root->right);
        print_data(root->data);
    }
}
```




```
void in_order(btn* root) {
    if (root != NULL){
        in_order(root->left);
        print_data(root->data);
        in_order(root->right);
    }
}
```




```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Recorridos




```
void pre_order(btn* root) {
    if (root != NULL){
        print_data(root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}
```

→ 20 10 15 30 25 22 27 35



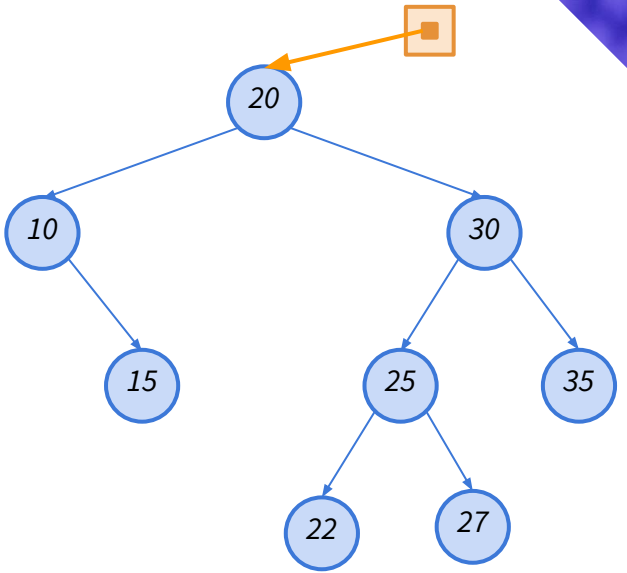

```
void post_order(btn* root) {
    if (root != NULL){
        post_order(root->left);
        post_order(root->right);
        print_data(root->data);
    }
}
```

→ 15 10 22 27 25 35 30 20




```
void in_order(btn* root) {
    if (root != NULL){
        in_order(root->left);
        print_data(root->data);
        in_order(root->right);
    }
}
```

→ 10 15 20 22 25 27 30 35




```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```



Árbol Binario de Búsqueda



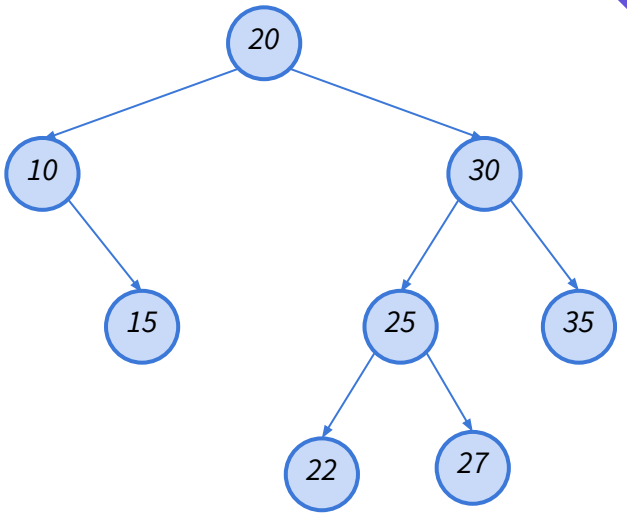
Árbol Binario de Búsqueda

Binary Search Tree

Para todo nodo

Todos los descendientes de la rama izquierda tienen un valor menor, y

Todos los descendientes de la rama derecha tienen un valor mayor

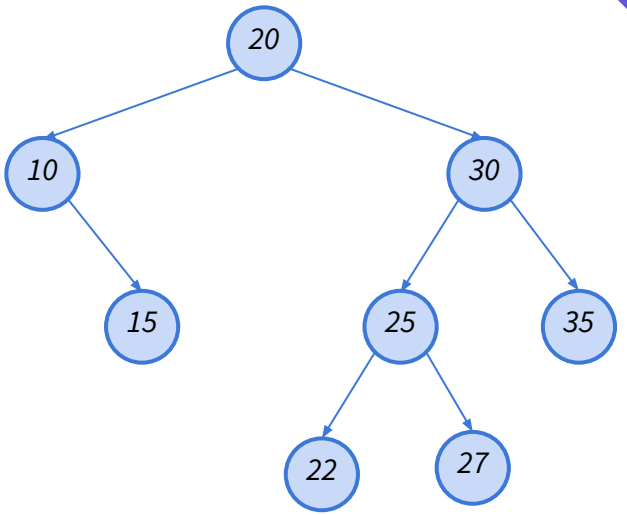



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
int contains(btn* node, int data) {
    while ((node != NULL) && (node->data != data)) {
        if (data < node->data)
            node = node->left;
        else
            node = node->right;
    }
    return node != NULL;
}
```

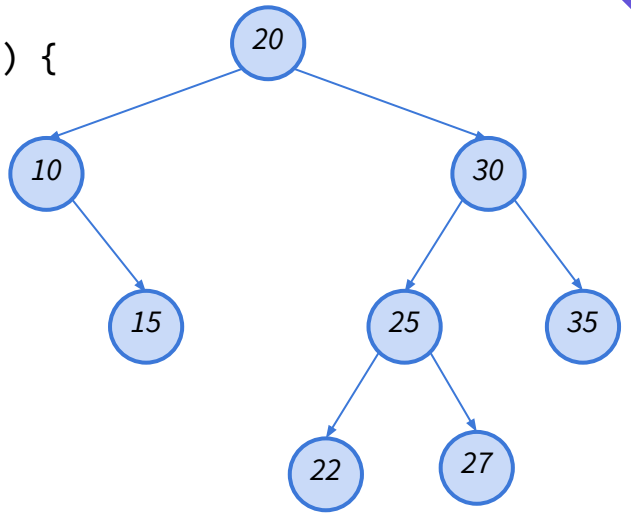



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
int contains(btn* node, t_data data, int cmp(t_data, t_data)) {
    while ((node != NULL) && (cmp(node->data, data) != 0)) {
        if (cmp(data, node->data) < 0)
            node = node->left;
        else
            node = node->right;
    }
    return node != NULL;
}
```

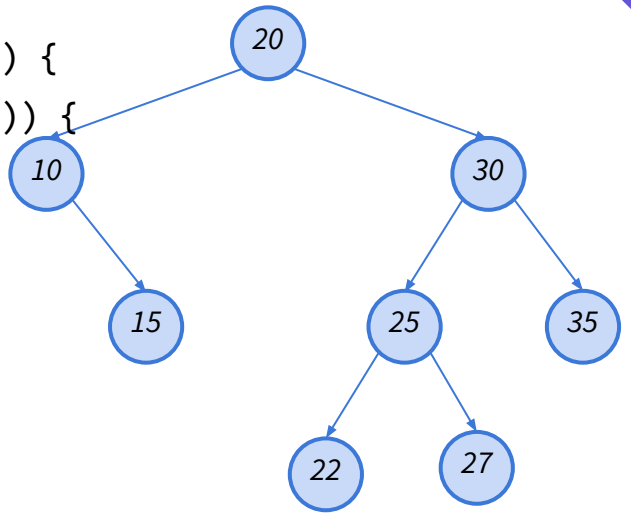



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
btn** find (btn** node, t_data data, int cmp(t_data, t_data)) {
    while ((*node != NULL) && (cmp((*node)->data, data) != 0)) {
        if (cmp(data, (*node)->data) < 0)
            node = &(*node)->left;
        else
            node = &(*node)->right;
    }
    return node;
}
```

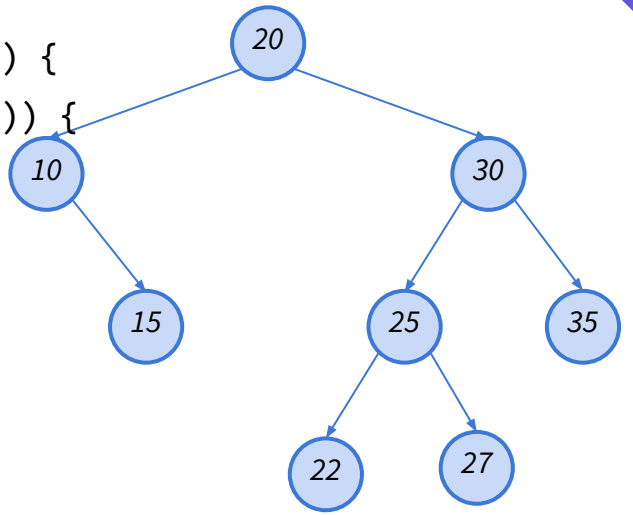



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
btn** find (btn** node, t_data data, int cmp(t_data, t_data)) {
    while ((*node != NULL) && (cmp((*node)->data, data) != 0)) {
        if (cmp(data, (*node)->data) < 0)
            node = &(*node)->left;
        else
            node = &(*node)->right;
    }
    return node;
}
```




```
void main (){
    btn* root = NULL;
    root = create_node(20);
    ...
    btn** found = find(&root, 25, cmp_int);
}
```

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

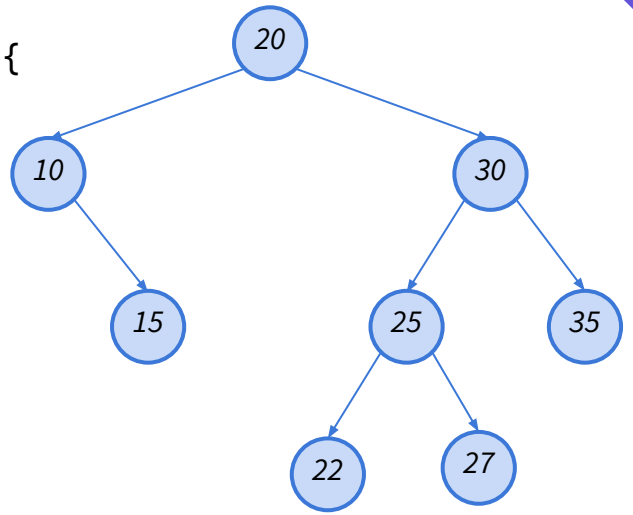
```
___ add(btn** root, btn* new_node, int cmp(t_data, t_data)) {



        btn** found = find(root, new_node->data, cmp);
        if (*found == NULL) {
            *found = new_node;

        }
    }

}
```

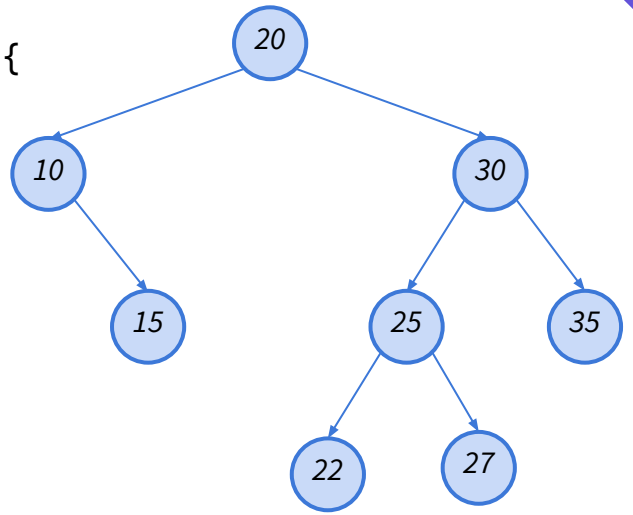



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
int add(btn** root, btn* new_node, int cmp(t_data, t_data)) {
    int res = 0;
    if (new_node != NULL) {
        btn** found = find(root, new_node->data, cmp);
        if (*found == NULL) {
            *found = new_node;
            res = 1;
        }
    }
    return res;
}
```

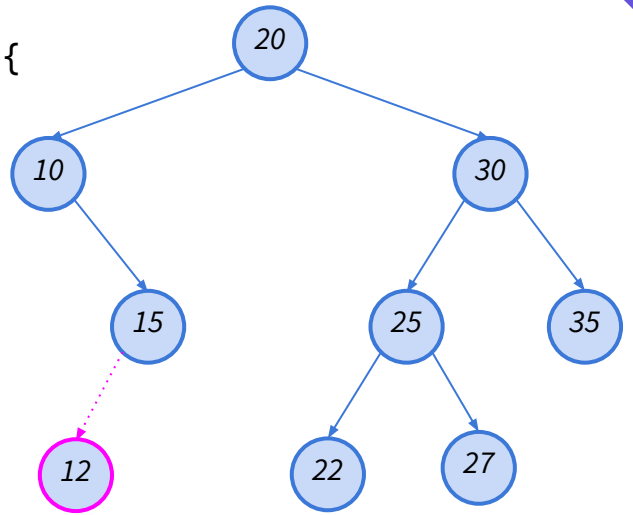



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
int add(btn** root, btn* new_node, int cmp(t_data, t_data)) {
    int res = 0;
    if (new_node != NULL) {
        btn** found = find(root, new_node->data, cmp);
        if (*found == NULL) {
            *found = new_node;
            res = 1;
        }
    }
    return res;
}
```



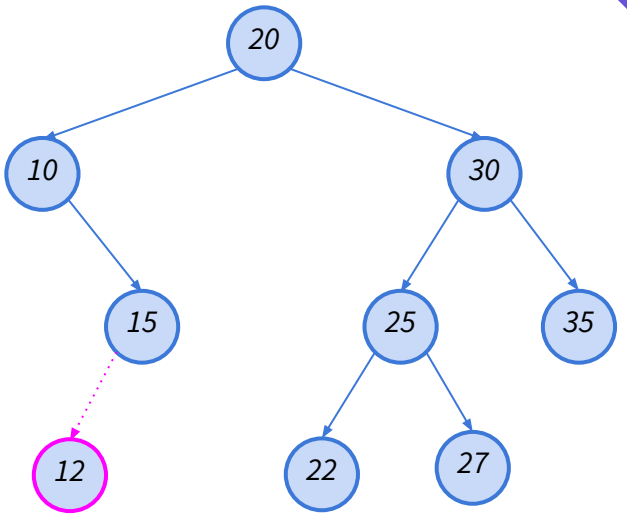

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

Casos:

1. Cuando el nodo es una hoja
2. Cuando el nodo tiene un solo hijo
3. Cuando el nodo tiene ambos hijos



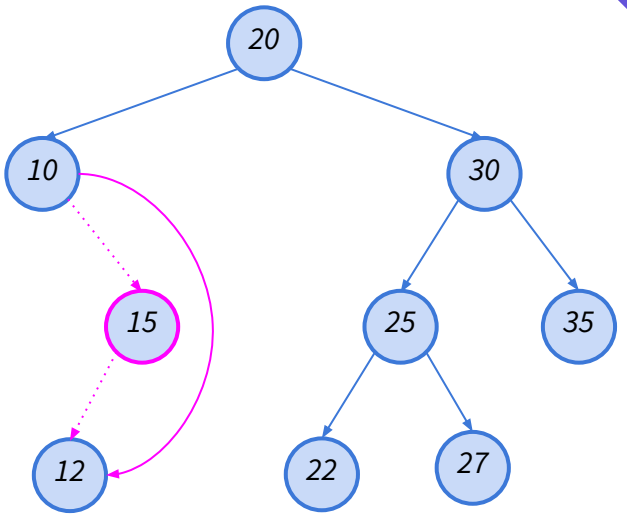

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

Casos:

1. Cuando el nodo es una hoja
2. Cuando el nodo tiene un solo hijo



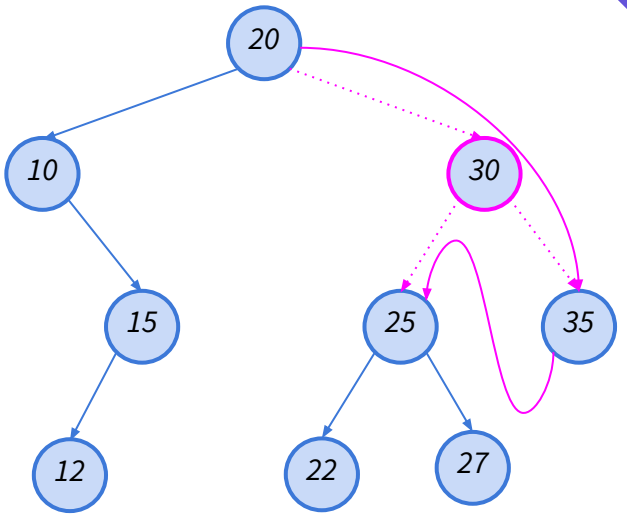

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

Casos:

1. Cuando el nodo es una hoja
2. Cuando el nodo tiene un solo hijo
3. Cuando el nodo tiene ambos hijos

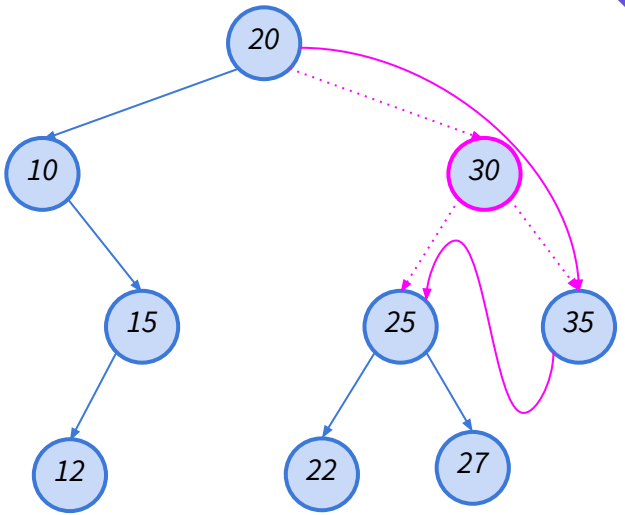



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

```
btn* rmv(btn** node, t_data data, int cmp(t_data, t_data)) {
    btn* res = NULL;
    btn** found = find(node, data, cmp);
    if (*found != NULL) {
        res = *found;
        add (&(*found)->right, (*found)->left, cmp);
        *found = (*found)->right;
        res->left = NULL;
        res->right = NULL;
    }
    return res;
}
```

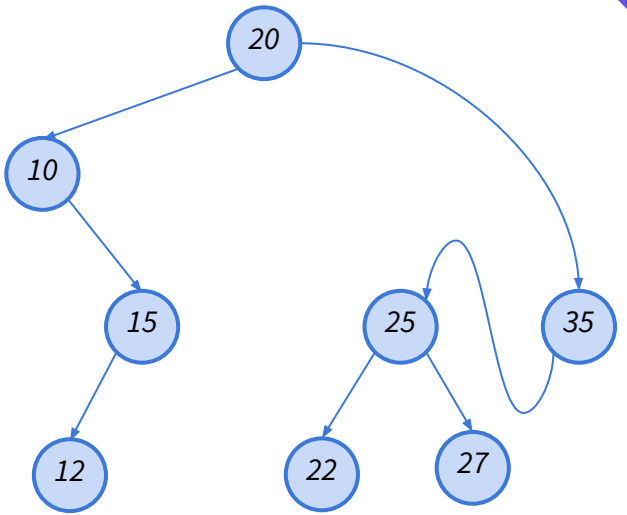



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

¿Y si eliminamos la raíz?

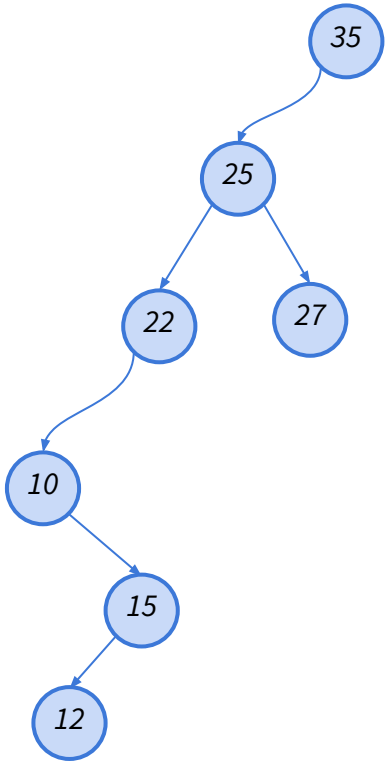


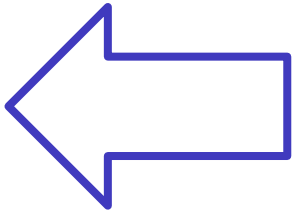

```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

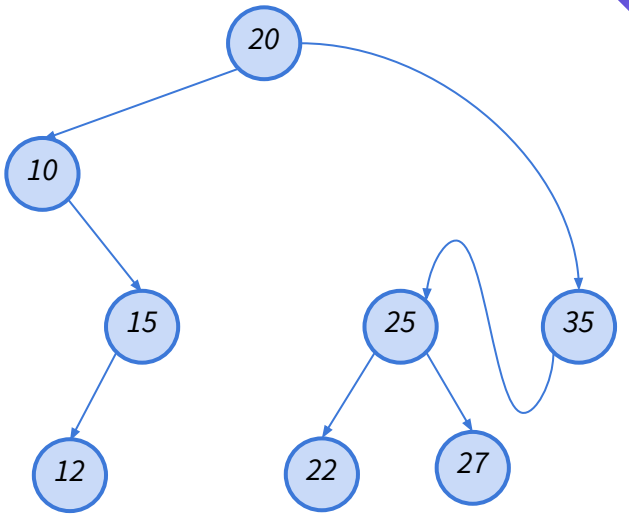
Árbol Binario de Búsqueda

Binary Search Tree

¿Y si eliminamos la raíz?





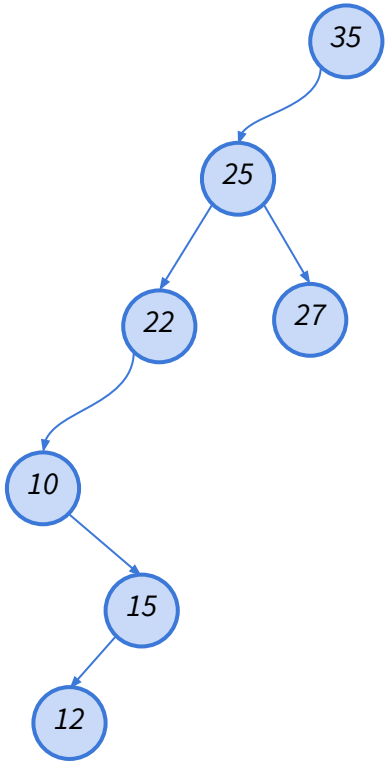



```
typedef struct _btn {
    t_data data;
    struct _btn* left;
    struct _btn* right;
} btn;
```

Árbol Binario de Búsqueda

Binary Search Tree

¿Y si eliminamos la raíz?

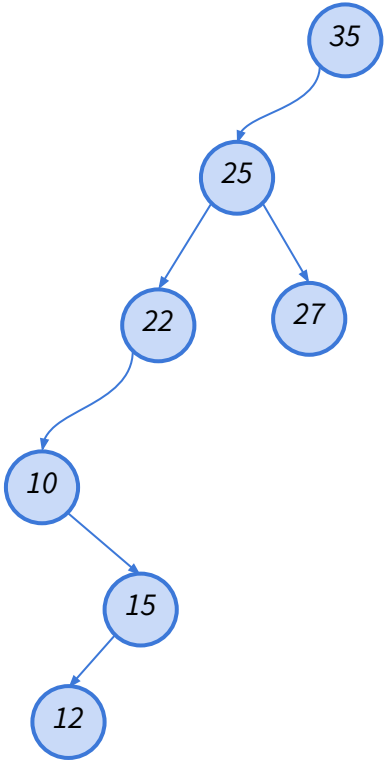


¿Y si el el orden de inserción fuera 10, 12, 15, 22, 25, 27, 35?

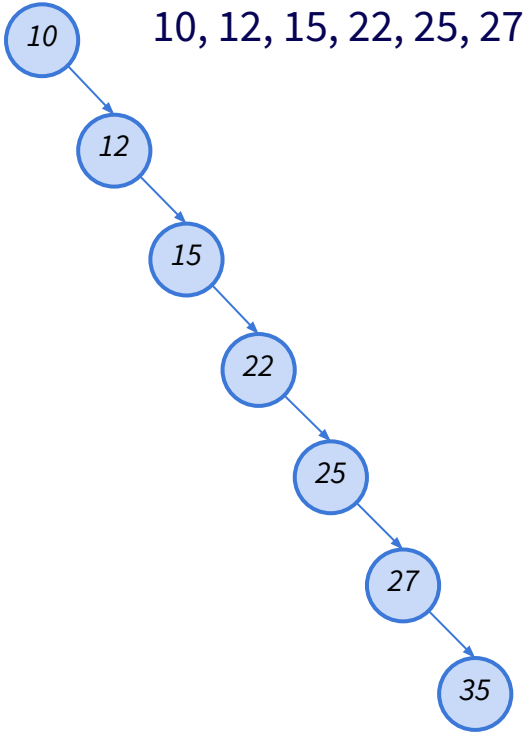
Árbol Binario de Búsqueda

Binary Search Tree

¿Y si eliminamos la raíz?



¿Y si el el orden de inserción fuera 10, 12, 15, 22, 25, 27, 35?




Continuará…